

Trabalho Final

Agente de Análise Exploratória de Dados em Linguagem Natural

Construção de um Agente LLM com *Tool Use* sobre arquivos CSV

Disciplina:	Tópicos Especiais em Ciência de Dados
Curso:	Tecnologia em Ciência de Dados
Semestre:	5º semestre
Modalidade:	Grupos de 2 a 4 alunos
Entrega:	A definir pelo professor

1 Apresentação

Este trabalho consolida competências desenvolvidas ao longo do semestre em processamento de linguagem natural, manipulação de dados e uso de modelos de linguagem de grande porte (LLMs). O objetivo é projetar e implementar um **agente conversacional** capaz de receber perguntas em português sobre um arquivo CSV e respondê-las executando, de forma autônoma, operações de análise exploratória de dados (EDA).

Diferentemente de um *chatbot* tradicional, um **agente** decide quais *ferramentas* (funções) chamar, em que ordem, e como compor o resultado em uma resposta coerente. Essa diferença é central no trabalho: o aluno não deve apenas gerar respostas, mas projetar um sistema que **raciocine sobre os dados** e **aja sobre eles** por meio de ferramentas Python pré-definidas.

Pergunta orientadora:

Dado um CSV qualquer e uma pergunta em linguagem natural (ex.: “qual a correlação entre idade e renda nos clientes do Sudeste?”), como construir um agente que escolha sozinho as operações pandas/estatísticas adequadas, execute-as, interprete os resultados e responda ao usuário com clareza?

2 Objetivos de Aprendizagem

Ao concluir o trabalho, o aluno deverá ser capaz de:

1. Projetar a arquitetura de um agente baseado em LLM com *tool use* (*function calling*).
2. Definir um conjunto coerente de ferramentas (*tools*) que cubram operações típicas de EDA: descrição estatística, agrupamento, correlação, contagem de valores, detecção de *outliers*, visualização.
3. Implementar o ciclo de execução do agente (*loop* de pensar → agir → observar) com tratamento de erros.
4. Construir um conjunto de avaliação (*benchmark*) com perguntas e respostas de referência.

5. Avaliar quantitativamente o desempenho do agente, incluindo análise de trajetórias (quais *tools* foram chamadas, em que ordem).
6. Discutir criticamente limitações, riscos (alucinação, custo, latência) e implicações éticas.

3 Escopo Técnico

3.1 O que o agente DEVE fazer

- Carregar um arquivo CSV indicado pelo usuário.
- Aceitar perguntas em português sobre os dados carregados.
- Decidir autonomamente quais ferramentas chamar para responder.
- Encadear chamadas a múltiplas ferramentas quando necessário (raciocínio em múltiplos passos).
- Lidar com erros das ferramentas (coluna inexistente, tipo incompatível, divisão por zero, etc.) sem travar.
- Apresentar respostas em linguagem natural, com números formatados e, quando aplicável, gráficos salvos como arquivos de imagem.
- Registrar (*logar*) cada passo da execução: ferramenta chamada, argumentos, resultado, decisão seguinte.

3.2 O que o agente NÃO precisa fazer

- Realizar modelagem preditiva (treinar regressões, classificadores, etc.).
- Aceitar múltiplos CSVs simultaneamente (escopo de um arquivo por sessão).
- Persistir memória entre sessões diferentes.
- Possuir interface web sofisticada (uma interface CLI ou Streamlit simples é suficiente).

3.3 Ferramentas mínimas obrigatórias

O agente deve oferecer, no mínimo, as seguintes *tools*:

1. `listar_colunas()` – retorna nomes e tipos de cada coluna.
2. `descrever_dados()` – equivalente a `df.describe()`, com extensão para colunas categóricas.
3. `contar_valores(coluna)` – distribuição de valores de uma coluna.
4. `filtrar(condicao)` – aplica filtro e retorna estatísticas do subconjunto.
5. `agrupar_e_agregar(grupo, coluna, funcao)` – `groupby` + `agg`.
6. `correlacao(coluna_a, coluna_b)` – correlação de Pearson ou Spearman.

7. `detectar_outliers(coluna)` – via IQR ou *z-score*.
8. `gerar_grafico(tipo, colunas)` – salva imagem (histograma, boxplot, scatter, barplot).

Critério de originalidade: grupos que adicionarem ferramentas além das obrigatórias e justificarem seu uso no relatório pontuam mais na rubrica.

4 Restrições Computacionais e Tecnológicas

Este trabalho foi projetado para ser executado em **notebooks comuns, sem GPU dedicada**. Não será aceito trabalho que dependa de *fine-tuning* de modelos ou de hardware acelerado para funcionar.

4.1 Stack recomendada

- **Linguagem:** Python 3.10+
- **LLM:** acesso via API a um dos seguintes:
 - Anthropic (Claude Haiku ou Sonnet)
 - OpenAI (GPT-4o-mini)
 - Google (Gemini Flash)
 - Modelo local via Ollama (Llama 3.2 3B, Qwen 2.5 3B) – aceitável, porém mais lento
- **Manipulação de dados:** pandas, numpy
- **Visualização:** matplotlib ou seaborn
- **Interface:** CLI (input/print) ou Streamlit
- **Orquestração:** *function calling* nativo da API **ou** framework leve (LangChain, LlamaIndex, Pydantic AI) – a escolha deve ser justificada no relatório

4.2 Sobre custos de API

Cada grupo deve usar créditos gratuitos disponibilizados pelos provedores em contas novas. O custo total esperado para o trabalho inteiro (desenvolvimento + avaliação) raramente passa de US\$ 2,00 por grupo, desde que sejam usados modelos das categorias *mini/flash/haiku*.

5 Conjunto de Dados

Cada grupo deverá escolher **um único** dataset entre as opções abaixo (ou propor outro, mediante aprovação do professor):

1. **Adult Income** (UCI) – dados socioeconômicos e renda.

2. **Titanic** (Kaggle) – passageiros e sobrevivência.
3. **COVID-19 Brasil** (Brasil.IO ou Ministério da Saúde) – casos por município/data.
4. **ENEM** (microdados públicos, amostra reduzida ≤ 50 mil registros).
5. **Dados de vendas** (qualquer dataset de *e-commerce* disponível no Kaggle).
6. **Spotify Tracks** – características musicais e popularidade.

O dataset escolhido deve ter, no mínimo, **6 colunas** (sendo ao menos 3 numéricas e 2 categóricas) e **1.000 registros**. Datasets muito pequenos não permitem avaliação significativa.

6 Avaliação do Agente

A avaliação do agente construído pelos alunos é **parte integrante do trabalho**, com peso significativo na nota. Não basta implementar – é preciso medir.

6.1 Construção do conjunto de avaliação

Cada grupo deverá construir um arquivo `benchmark.json` com **no mínimo 30 perguntas**, distribuídas em:

- **10 perguntas factuais simples** – exigem uma única *tool* (ex.: “*quantas linhas tem o dataset?*”).
- **15 perguntas analíticas** – exigem 2 ou mais *tools* em sequência (ex.: “*qual a renda média dos homens com mais de 40 anos?*”).
- **5 perguntas ambíguas ou inválidas** – o agente deve reconhecer a ambiguidade ou recusar (ex.: “*qual a melhor coluna?*”).

Para cada pergunta, o grupo deve registrar a **resposta de referência** (calculada manualmente em pandas).

6.2 Métricas obrigatórias

1. **Acurácia da resposta final**: a resposta numérica/textual do agente coincide com a de referência? (binário por pergunta, agregado em %).
2. **Taxa de execução bem-sucedida**: o agente terminou sem *crash*?
3. **Número médio de *tool calls* por pergunta** – mede eficiência.
4. **Latência média** – tempo do envio da pergunta até a resposta final.
5. **Custo médio por pergunta** (em US\$ ou em *tokens*).

6.3 Análise qualitativa obrigatória

Além das métricas, o relatório deve conter:

- Pelo menos **3 exemplos de trajetórias completas** (pergunta → tools → resposta), comentadas.
- Análise de **pelo menos 3 erros** cometidos pelo agente, com hipóteses sobre a causa.
- Discussão sobre **alucinação**: o agente alguma vez “inventou” uma resposta sem chamar *tools*?

7 Entregáveis

1. **Repositório Git** (GitHub, GitLab ou similar), público ou compartilhado com o professor, contendo:
 - Código-fonte completo, organizado em módulos.
 - Arquivo `README.md` com instruções de instalação e execução.
 - Arquivo `requirements.txt`.
 - Dataset utilizado (ou *link* para ele, se for grande).
 - Arquivo `benchmark.json` com as 30+ perguntas e respostas de referência.
 - *Logs* de pelo menos uma execução completa do *benchmark*.
2. **Relatório em PDF** (até 12 páginas, sem contar referências), contendo:
 - Introdução e justificativa do problema.
 - Descrição do dataset escolhido.
 - Arquitetura do agente (com diagrama).
 - Decisões de implementação justificadas.
 - Resultados quantitativos da avaliação.
 - Análise qualitativa (trajetórias, erros, alucinação).
 - Discussão crítica (limitações, custo, ética).
 - Conclusão e trabalhos futuros.
3. **Apresentação ao vivo** (15 minutos + 5 de perguntas) com **demonstração funcional** do agente.

8 Cronograma Sugerido

Semana	Atividade
1	Formação dos grupos, leitura do enunciado, escolha do dataset e do LLM.
2	Análise exploratória “manual” do dataset escolhido. Definição das <i>tools</i> a implementar.
3	Implementação das <i>tools</i> (Python puro, sem agente ainda). Testes unitários.
4	Integração com a API do LLM e <i>loop</i> do agente. Primeiro fluxo <i>end-to-end</i> .
5	Construção do <code>benchmark.json</code> . Refinamento do agente com base em erros encontrados.
6	Execução completa do <i>benchmark</i> . Coleta de métricas. Análise de erros.
7	Redação do relatório. Preparação da apresentação.
8	Apresentações.

9 Rubrica de Avaliação

Dimensão	Peso
Definição do escopo e do dataset (qualidade da motivação e da escolha)	10%
Design do agente (<i>tools</i> , arquitetura, fluxo de decisão)	20%
Implementação (qualidade do código, modularização, tratamento de erros)	20%
Conjunto de avaliação bem construído (30+ perguntas, bem distribuídas)	15%
Resultados quantitativos + análise de trajetórias do agente	15%
Análise crítica (custo, latência, falhas, alucinação, ética)	10%
Demonstração funcional e relatório (clareza, organização, profundidade)	10%
Total	100%

9.1 Critérios por nível de entrega

Nível	Descrição
Insuficiente (0–4)	Agente não funciona ou funciona apenas em casos triviais. <i>Benchmark</i> ausente ou com menos de 15 perguntas. Relatório superficial.
Regular (5–6)	Agente funciona para perguntas simples mas falha frequentemente nas analíticas. <i>Benchmark</i> mínimo. Pouca análise de erros.
Bom (7–8)	Agente funciona consistentemente. <i>Benchmark</i> bem construído. Análise de erros presente, mas com pouca profundidade.
Excelente (9–10)	Agente robusto, com tratamento de erros e <i>tools</i> além das obrigatórias. <i>Benchmark</i> rico e bem distribuído. Análise crítica madura, discutindo alucinação, custo e implicações éticas.

10 Política de Integridade Acadêmica

É **permitido** o uso de LLMs (ChatGPT, Claude, Gemini etc.) como assistentes de programação durante o desenvolvimento. **Não é permitido** entregar código que o grupo não compreenda. Durante a apresentação, qualquer integrante pode ser questionado sobre qualquer linha do código.

Cópia de código entre grupos resulta em nota zero para todos os envolvidos.

11 Perguntas Frequentes

Posso usar LangChain/LlamaIndex?

Sim, desde que você compreenda o que o framework está fazendo. Frameworks aceleram o desenvolvimento mas escondem detalhes importantes que podem ser cobrados na apresentação.

E se o LLM “alucinar” uma coluna que não existe?

Esse é exatamente o tipo de comportamento que sua avaliação deve capturar. O agente deve idealmente recusar ou pedir esclarecimento.

Posso usar um modelo local via Ollama?

Sim. A penalidade é a latência (várias vezes mais lenta) e a qualidade inferior em raciocínio multi-passo. Considere usar para desenvolvimento e API para a avaliação final.

Posso fazer fine-tuning?

Não. O escopo do trabalho é construção de agente, não treinamento de modelo.

Bom trabalho.